

- Installer **Java JDK** compatible avec votre projet; le `pom.xml` cible Java 21, alors que le README mentionne JDK 11 ou plus, donc le plus sûr est d'utiliser **Java 21** pour éviter les écarts de compilation.
- Installer **MySQL Server** et créer la base de données attendue par l'application, nommée `BANK`.
- Disposer de **JavaFX** et du pilote **MySQL Connector/J**; le `pom.xml` déclare les dépendances `JavaFX` et `mysql-connector-j` version 8.4.0.

Configuration JDBC

La connexion JDBC est centralisée dans `DatabaseConnection.java` avec les paramètres suivants: URL `jdbc:mysql://localhost:3306/BANK`, utilisateur `root`, mot de passe `admin`.

Pour adapter la connexion à votre machine, modifier ces constantes dans `DatabaseConnection.java`:

```
private static final String URL = "jdbc:mysql://localhost:3306/BANK";
private static final String USER = "root";
private static final String PASSWORD = "admin";
```

Cette classe ouvre la connexion via `DriverManager.getConnection(...)` et la réutilise ensuite si elle est encore active.

Initialiser la base

- Démarrer MySQL.
- Créer la base `BANK` si elle n'existe pas.
- Exécuter le script `DB.sql` pour créer les tables nécessaires.

Si vous voulez aussi insérer des données de test, le README indique qu'il faut utiliser `DbManager.java` en décommentant la méthode de test prévue pour peupler la base avec des utilisateurs comme `admin`, `client1` et `client2`.

Compiler le projet

Si vous utilisez **Maven**, la compilation standard se fait depuis la racine du projet avec:

```
mvn clean compile
```

Cette commande est cohérente avec la présence du `pom.xml` , qui configure le compilateur Maven ainsi que les dépendances JavaFX et MySQL. Si Maven n'est pas utilisé dans votre environnement, il faut alors compiler manuellement avec `javac` , mais comme le projet est modulaire (`module-info.java`) et dépend de JavaFX ainsi que du connecteur MySQL, Maven reste l'option la plus simple.

Démarrer le serveur

Le README indique que le serveur RMI se lance avec :

```
javac project/bank/server/ServerLauncher.java
java project.bank.server.ServerLauncher
```

Cette étape démarre la partie serveur chargée d'exposer le service bancaire distant via RMI. Le README précise aussi que la configuration RMI attend un registre sur `localhost` , port `1099` , avec un service nommé `BanqueService` .

Démarrer le client

Le point d'entrée client visible dans votre code est `Launcher.java` , qui lance l'application JavaFX via `Application.launch(HelloApplication.class, args)` .

D'après le README, le client se lance avec :

```
java project.bank.MAIN.Launcher
```

Cette commande ouvre l'interface JavaFX de l'application bancaire.

Ordre recommandé de démarrage

1. Démarrer **MySQL**.
2. Vérifier la configuration dans `DatabaseConnection.java` .
3. Exécuter `DB.sql` pour créer la base et les tables.
4. Compiler le projet avec `mvn clean compile` .
5. Lancer le **serveur RMI**.
6. Lancer le **client JavaFX**.

Exemple résumé

```
# 1) Compiler
mvn clean compile

# 2) Lancer le serveur
java project.bank.server.ServerLauncher

# 3) Lancer le client
java project.bank.MAIN.Launcher
```

Ces commandes correspondent à la logique générale décrite dans le README, même si le projet semble encore contenir quelques incohérences de configuration entre la documentation et certains fichiers Maven; pour un rendu académique, vous pouvez signaler que Maven est utilisé pour gérer les dépendances et que le lancement effectif repose sur un serveur RMI puis un client JavaFX.

Remarque importante

Il existe un **décalage** entre certains éléments du README et du `pom.xml`, notamment sur la version de Java et sur la classe principale configurée dans le plugin JavaFX Maven, ce qui suggère que la documentation et la configuration Maven ne sont pas totalement synchronisées.